

MAJOR PROJECT

Smart Retail & Customer Intelligence Platform

1. Project Overview

Title: AI-Powered Smart Retail & Customer Intelligence Platform

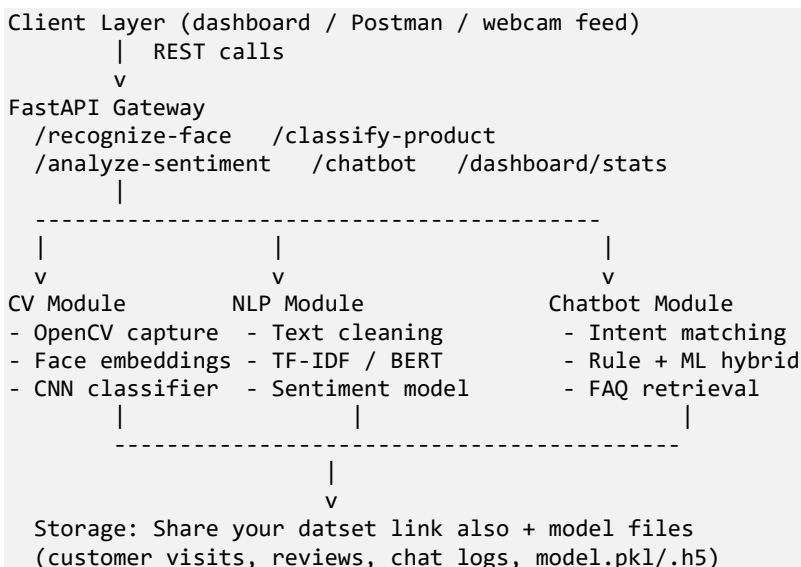
Goal: Build a single deployed system that a retail/e-commerce business could realistically use — it recognizes returning customers via face recognition, analyzes customer feedback/chat sentiment, answers FAQs via a chatbot, and exposes everything through a production-style API.

This project is designed so every Week 6 syllabus topic maps directly to a module:

Syllabus Topic	Project Module
OpenCV basics	Image preprocessing, webcam/video capture
Image classification	Product category classifier
Face recognition	Customer recognition & visit logging
Text preprocessing	Cleaning reviews/chat text
Sentiment analysis	Customer review/feedback classifier
Chatbot basics	FAQ/support chatbot (rule-based + ML)
ML pipelines	Unified pipeline combining all models
Pickle / Joblib	Model serialization
Flask / FastAPI	REST API serving all models
API deployment	Dockerized deployment to cloud

2. System Architecture

High-level flow of the platform:



3. Module-by-Module Build Plan

Module A — Computer Vision

A1. OpenCV Basics

- Capture webcam/video feed, apply grayscale, resize, blur, edge detection (Canny), face bounding boxes with Haar Cascades.
- Deliverable: `cv_utils.py` with reusable preprocessing functions.

A2. Image Classification

- Dataset: a small product-category dataset (e.g., Fashion-MNIST, or a scraped 5-class product image set: shoes, bags, electronics, clothing, groceries).
- Model: Transfer learning with MobileNetV2 (fast, deployable) using TensorFlow/Keras or PyTorch.
- Output: predicted category + confidence score.
- Deliverable: `product_classifier.h5` (or `.pt`).

A3. Face Recognition Fundamentals

- Use `face_recognition` (dlib-based) or OpenCV's LBPH recognizer for simplicity.
- Pipeline: detect face → generate encoding → compare against stored customer encodings → log visit with timestamp.
- Use case: "returning customer" detection for loyalty analytics — using a demo dataset of consenting sample faces (e.g., LFW subset).
- Deliverable: `face_db.pkl` (encodings) + `face_recognition_module.py`.

Ethics note to include in the report: discuss consent, data privacy, and bias considerations of facial recognition in retail — this is expected in an "industry" project write-up and often scored by evaluators.

Module B — Natural Language Processing (NLP)

B1. Text Preprocessing

- Lowercasing, punctuation removal, stopword removal, lemmatization (spaCy/NLTK), tokenization.
- Apply to a reviews dataset (e.g., Amazon Reviews subset, or Kaggle "Women's E-Commerce Clothing Reviews").

B2. Sentiment Analysis

- Baseline: TF-IDF + Logistic Regression/SVM.
- Upgrade option: fine-tune a small transformer (DistilBERT via HuggingFace) for higher accuracy.
- Output: Positive / Negative / Neutral + confidence.
- Deliverable: `sentiment_model.pkl` + `vectorizer.pkl`.

B3. Chatbot Basics

- Hybrid approach: rule-based intent matching for FAQs (order status, return policy, store hours).
- ML-based fallback: intent classification model (TF-IDF + classifier) trained on a small custom intents JSON.
- Deliverable: `intents.json`, `chatbot_model.pkl`.

Module C — ML Pipeline & Deployment

C1. Unified Pipeline

- Wrap all three models (face recognition, image classifier, sentiment/chatbot) behind a single `pipeline.py` that loads models once at startup.

C2. Serialization

- Use `joblib` for sklearn models, native `.h5/.pt` save for deep learning models, `pickle` for encodings/intents.

C3. API Layer (FastAPI recommended over Flask for this project)

- POST /recognize-face — upload image → returns customer ID/status
- POST /classify-product — upload image → returns category
- POST /analyze-sentiment — text → returns sentiment
- POST /chatbot — message → returns bot reply
- GET /dashboard/stats — returns aggregate visit/sentiment stats (JSON, for a simple frontend chart)
- Use Pydantic models for request/response validation.
- Add Swagger docs (auto-generated by FastAPI at /docs).

C4. Deployment

- Containerize with Docker (Dockerfile + requirements.txt).
- Deploy to Render / Railway / AWS EC2 / Google Cloud Run (free-tier friendly options for students).
- Add basic auth or API key header for the endpoints to simulate production security.
- Optional: CI/CD via GitHub Actions (lint + test + build image on push).

4. Suggested Folder Structure

```
smart-retail-ai/
├── app/
│   ├── main.py                # FastAPI entrypoint
│   ├── routers/
│   │   ├── vision.py
│   │   ├── nlp.py
│   │   └── chatbot.py
│   ├── models/
│   │   ├── product_classifier.h5
│   │   ├── face_db.pkl
│   │   ├── sentiment_model.pkl
│   │   └── chatbot_model.pkl
│   ├── services/
│   │   ├── cv_service.py
│   │   ├── nlp_service.py
│   │   └── chatbot_service.py
│   └── schemas.py
├── notebooks/                 # training/experiment notebooks
│   ├── 01_image_classifier_training.ipynb
│   ├── 02_face_recognition_setup.ipynb
│   └── 03_sentiment_model_training.ipynb
├── data/
│   ├── reviews.csv
│   └── intents.json
├── tests/
│   └── test_endpoints.py
├── requirements.txt
├── README.md
└── .github/workflows/deploy.yml
```

5. Timeline (9-Days Sprint)

Days	Task
1–2	Data collection, EDA, preprocessing (both CV & NLP datasets)
3–4	Train & evaluate image classifier; set up face recognition
5–6	Train sentiment model; build chatbot intents + model

Days	Task
6	Build unified pipeline; serialize all models
7	Build FastAPI endpoints + Gradio docs + input validation
8	Write automated tests
8	Build a minimal frontend/dashboard (optional: Streamlit or React)
9	Final report, architecture diagram, demo video, README

6. Evaluation Criteria (Rubric)

Criteria	Weight
Model accuracy (each module)	25%
Code quality & pipeline design	20%
API design & documentation	15%
Deployment (working live/Docker demo)	20%
Report: architecture, ethics, tradeoffs	10%
Presentation/demo	10%

7. Stretch Goals (for advanced learners)

- Replace TF-IDF sentiment model with a fine-tuned DistilBERT.
- Add a real-time video stream endpoint (WebSocket) for live face recognition.
- Add model monitoring (log prediction confidence drift over time).
- Add a simple Streamlit dashboard showing visit trends + sentiment trends.
- A/B test two chatbot response strategies and log which gets better user ratings.

8. Recommended Datasets (all free)

- Face recognition demo: LFW (Labeled Faces in the Wild) — for practice only, or self-collected consenting sample images.
- Product images: Fashion-MNIST, or Kaggle "Retail Product Checkout Dataset."
- Reviews/sentiment: Kaggle "Women's E-Commerce Clothing Reviews," or Amazon Reviews (subset).
- Chatbot intents: Build a custom intents.json (20–30 intents covering FAQs) — this is standard practice and shows original work.

9. Why This Project Works as an "Industry" Capstone

- It mirrors real retail-tech stacks (in-store analytics + customer support automation), a common theme in current retail AI deployments.
- It forces integration across CV + NLP + MLOps rather than three disconnected mini-projects.
- The deployment step (Docker + cloud + API docs) is exactly what's expected in junior ML engineer interviews and portfolio reviews.

- Ethics/privacy discussion around face recognition is increasingly expected in real deployments and interview conversations.